

Diffusion-Based Methods for Inverse Problems in Physical Data

Skolkovo Institute of Science and Technology
Deep Learning Final Project

Students: Mariia Abdullaeva, Andrei Efimov,
Anton Losev, Amir Sadreev

Skoltech

github.com/mabd11v/DL_final_project

- 1 Introduction & Motivation
- 2 Background: Diffusion Models
- 3 Dataset & Model
- 4 Conditional Sampling Methods
- 5 Physics-Informed Sampling
- 6 Evaluation
- 7 Results
- 8 Conclusion

Inverse Problems in Physics

The core challenge

In many physical systems the full state $\mathbf{u}$ cannot be measured directly. Sensors are sparse, expensive, or obscured. We observe only

$$\mathbf{y} = A(\mathbf{u}) + \boldsymbol{\eta},$$

and must recover the full field $\mathbf{u}$ from incomplete, possibly noisy data.

Practical examples

- **Fluid dynamics:** PIV measurements give velocity at a sparse grid; the rest of the flow field is unknown.
- **Meteorology:** weather stations cover a tiny fraction of the atmosphere.
- **Medical imaging:** MRI/CT measure a transform of the tissue density, not density itself.

Why is it hard?

The problem is ill-posed: many fields $\mathbf{u}$ are consistent with the same observation $\mathbf{y}$. A good reconstruction method must encode a *prior* over physically realistic fields.

Classical Approaches and Their Limits

Standard baselines

- **Interpolation** (bilinear, RBF): cheap, but ignores global structure and physical constraints.
- **Variational / PDE-based methods**: require hand-crafted regularisers and domain knowledge.
- **Compressed sensing**: requires sparsity in a fixed basis; struggles with complex turbulent fields.

What is missing

- No uncertainty quantification — outputs a single point estimate.
- Prior is fixed by hand, not learned from data.
- Cannot adapt to different observation operators without redesign.

Our approach

Use a **diffusion model** trained on clean physical fields as a *data-driven prior*. At inference time, measurement guidance steers the sampling toward reconstructions that are both physically plausible *and* consistent with the observations. No retraining is needed when A changes.

Project Scope and Goals

Setting

- **Data:** 2D Kolmogorov flow — a canonical turbulent velocity field (u_x, u_y) .
- **Task:** single-state reconstruction (not forecasting).
- **Operators A :** four representative types (sparse, masked, downsampled, blurred).

Methods evaluated

- 1 **DDNM** — null-space decomposition for linear operators.
- 2 **DPS** — gradient-guided posterior sampling (general differentiable A).
- 3 **RePaint** — observation pasting for inpainting.
- 4 **DDRM** — spectral decomposition via SVD/Fourier.

Physics constraint

Incompressibility $\nabla \cdot \mathbf{u} = 0$ is enforced via Helmholtz projection during sampling, at zero additional training cost.

Variance-Preserving SDE (VP-SDE)

Clean field $\mathbf{x}_0$ is gradually corrupted by adding Gaussian noise:

$$\mathbf{x}_t = \mu_t \mathbf{x}_0 + \sigma_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I),$$

with schedule

$$\mu_t = \cos(\omega t), \quad \sigma_t = \sqrt{1 - \mu_t^2}.$$

At $t = 0$: $\mathbf{x}_0$ is the clean field. At $t = T$: $\mathbf{x}_T \approx \mathcal{N}(0, I)$.

Score function

The score $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)$ parameterises the reverse process. We train a neural network $\varepsilon_\theta(\mathbf{x}_t, t)$ to predict the noise ε , which is equivalent to learning the score:

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \varepsilon} \left\| \varepsilon_\theta(\mathbf{x}_t, t) - \varepsilon \right\|^2.$$

Score-Based Diffusion: Reverse Process

Deterministic reverse sampler

Starting from $\mathbf{x}_T \sim \mathcal{N}(0, I)$, iterate from $t = T$ to 0:

- 1 Predict clean field: $\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sigma_t \varepsilon_\theta(\mathbf{x}_t, t)}{\mu_t}$.
- 2 Step to next time: $\mathbf{x}_{t-1} = \mu_{t-1} \hat{\mathbf{x}}_0 + \sigma_{t-1} \varepsilon_\theta(\mathbf{x}_t, t)$.

Why diffusion for inverse problems?

- The learned prior covers the entire distribution of realistic fields.
- Conditional sampling can be plugged in at inference time without modifying the score model.
- Multiple samples yield a distribution over plausible reconstructions.

Key idea for conditional sampling

At each reverse step, after computing $\hat{\mathbf{x}}_0$, **inject measurement information** to enforce $A(\hat{\mathbf{x}}_0) \approx \mathbf{y}$. Different methods differ in how this injection is done.

Kolmogorov Flow Dataset

Physical setup

Kolmogorov flow: 2D incompressible Navier-Stokes with periodic forcing.

$$\partial_t \omega + (\mathbf{u} \cdot \nabla) \omega = \nu \Delta \omega + f, \quad f = A \sin(k_f y).$$

Simulation in vorticity form on a periodic 256×256 grid.

Dataset construction

- Velocity field (u_x, u_y) computed from vorticity via stream function.
- Average-pooled to 64×64 : tensor shape $[N, 2, 64, 64]$.
- Stored as .npz with metadata: viscosity, drag, forcing_amp, split.
- Splits: **train** / **val** / **test** by trajectory ID.

Normalisation

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x}_{\text{raw}} - \mu_{\text{train}}}{\sigma_{\text{train}}}.$$

All sampling runs in normalised space; metrics are computed in raw velocity space.

Score Model Architecture

Model

VP-SDE score network: `diffusers.UNet2DModel` with

- **Circular padding** to respect periodic boundary conditions.
- **Fourier coordinate channels:** $\sin(x)$, $\cos(x)$, $\sin(y)$, $\cos(y)$ appended to input. These are clean conditioning channels — they are not noised and do not enter the loss.
- Input: $[\mathbf{x}_t, \sin x, \cos x, \sin y, \cos y]$, 6 channels total.

Training objective

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \varepsilon} \left\| \varepsilon_{\theta}(\mathbf{x}_t, t) - \varepsilon \right\|^2.$$

Noise schedule: cosine VP-SDE. Checkpoint saves `mean` and `std` for normalisation.

Unconditional sampling quality

The trained prior generates velocity snapshots that are visually and statistically indistinguishable from held-out Kolmogorov fields. Divergence $\|\nabla \cdot \mathbf{u}\|$ of prior samples is small but not exactly zero — physics constraint is added separately.

Model Architecture & Training Setup

UNet architecture (`diffusers.UNet2DModel`)

Property	Value
Input channels	6 (u_x, u_y noisy + $\sin x, \cos x, \sin y, \cos y$ clean)
Output channels	2 (predicted noise $\hat{\epsilon}$)
Spatial resolution	64×64
Channel widths	$96 \rightarrow 192 \rightarrow 384$ (3 levels)
Residual blocks / level	3
Attention	AttnDown/UpBlock2D on levels 2–3; head dim 32
Activation / norm	SiLU / GroupNorm-32
Conv padding	circular (periodic boundary conditions)

Training setup

- **Hardware:** NVIDIA A100 GPU; TF32 enabled (`allow_tf32, float32_matmul_precision="high"`).
- **Precision:** **BF16** (`torch.amp.autocast` with `bfloat16`); sampling runs in FP32 for numerical stability.
- **Optimiser:** AdamW, $\text{lr} = 2 \times 10^{-4}$, weight decay 10^{-3} , linear LR decay to 0 over

Observation Operators

Four fixed operators (applied identically to both channels)

Operator	Definition	Reduction
Regular sparse grid	$M_{ij} = 1$ if $i \bmod 4 = 0, j \bmod 4 = 0$	$\approx 6\%$ known
Central box mask	$\mathbf{u}$ known on $[16:48, 16:48]$	25% known
Average downsample	4×4 avg-pool: $[2, 64, 64] \rightarrow [2, 16, 16]$	6.25%
Periodic Gaussian blur	circular conv., $\sigma=2$ + Gaussian noise σ_y	full size, degraded

Noisy observation model

For all operators:

$$\mathbf{y} = A(\mathbf{u}) + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(0, \sigma_y^2 I).$$

$\sigma_y > 0$ for blur; $\sigma_y = 0$ (noiseless) for sparse grid, box mask, and downsample.

DDNM: Null-Space Decomposition

Idea

For any linear A , decompose $\hat{\mathbf{x}}_0$ into *range* and *null-space* components. Replace the range part with the pseudoinverse of the observation.

Correction step

$$\hat{\mathbf{x}}_0^{\text{corr}} = A^\dagger \mathbf{y} + (I - A^\dagger A) \hat{\mathbf{x}}_0.$$

For a mask operator $A(\mathbf{x}) = M \odot \mathbf{x}$, this reduces to pixel substitution:

$$\hat{\mathbf{x}}_0^{\text{corr}} = M \odot \mathbf{y}_{\text{known}} + (1 - M) \odot \hat{\mathbf{x}}_0.$$

For noisy observations, use a soft update with step size λ_t :

$$\hat{\mathbf{x}}_0^{\text{corr}} = \hat{\mathbf{x}}_0 + \lambda_t A^\dagger (\mathbf{y} - A \hat{\mathbf{x}}_0).$$

Reverse step

VP reverse step proceeds from $\hat{\mathbf{x}}_0^{\text{corr}}$, not the original $\hat{\mathbf{x}}_0$. Applicable to: sparse grid, box mask, downsample, blur.

DPS: Diffusion Posterior Sampling

Idea

No pseudoinverse required. Compute a gradient correction directly through the predicted $\hat{\mathbf{x}}_0$. Works for any *differentiable* operator A .

Algorithm

At each reverse step with differentiable $\mathbf{x}_t$:

- 1 Predict: $\hat{\mathbf{x}}_0 = \hat{\mathbf{x}}_0(\mathbf{x}_t, t)$ (keep graph for autograd).
- 2 Measurement loss: $\mathcal{L}_y = \|A(\hat{\mathbf{x}}_0) - \mathbf{y}\|_2^2$.
- 3 Gradient: $\mathbf{g} = \nabla_{\mathbf{x}_t} \mathcal{L}_y$.
- 4 Corrected state: $\mathbf{x}_t^{\text{corr}} = \mathbf{x}_t - s_t \mathbf{g}$.
- 5 Proceed with VP reverse step from $\mathbf{x}_t^{\text{corr}}$.

Model weights are **not updated**; no optimiser is needed.

Key hyperparameters

guidance_scale s_t , measurement_sigma σ_y , guidance_start/end — time window where guidance is active, gradient clipping for stability.

RePaint: Observation Pasting

Idea (mask operators only)

At every reverse step, *paste* the observation noised to the current diffusion level into the known region; let the prior generate the unknown region.

Algorithm

$$\mathbf{x}_t^{\text{known}} = \mu_t \mathbf{y} + \sigma_t \varepsilon, \quad \mathbf{x}_t = M \odot \mathbf{x}_t^{\text{known}} + (1 - M) \odot \mathbf{x}_t^{\text{generated}}.$$

Jumps

After each merge, the sampler can *jump back* to a noisier state and repeat several reverse steps. This helps the model reconcile the generated region with the boundary of the known region.

Limitation

RePaint is only valid for mask/inpainting operators. **Not applicable** to downsample or blur.

DDRM: Denoising Diffusion Restoration

Idea

For operators with a convenient spectral structure, decompose via SVD:

$$A = U\Sigma V^\top.$$

Work component-by-component in this basis: mix the prior prediction and the measurement separately at each singular value, weighted by Σ_i and σ_y .

Periodic blur

Circular convolution diagonalises in the Fourier basis. The blur operator acts as multiplication by transfer function $H(k)$:

$$\hat{\mathbf{y}}(k) = H(k) \hat{\mathbf{u}}(k) + \hat{\boldsymbol{\eta}}(k).$$

Correction per frequency k :

$$\hat{\mathbf{x}}_0^{\text{corr}}(k) = \frac{H(k) \hat{\mathbf{y}}(k) / \sigma_y^2 + \hat{\mathbf{x}}_0(k) / \sigma_t^2}{H(k)^2 / \sigma_y^2 + 1 / \sigma_t^2}.$$

Applicability

Primary targets: **downsample** and **blur**. Applied after DDNM and DPS.

Methods vs. Observation Operators

Method	Sparse grid	Box mask	Downsample	Blur
DDNM	✓	✓	✓	✓
DPS	✓	✓	✓	✓
RePaint	✓	✓	—	—
DDRM	(secondary)	(secondary)	✓	✓

Rationale

- **DDNM** and **RePaint**: most natural for mask/inpainting settings.
- **DPS**: universal differentiable baseline; runs on all four operators.
- **DDRM**: reserved for operators with a convenient spectral representation.

Divergence-Free Constraint

Physical constraint

Kolmogorov flow is incompressible:

$$\nabla \cdot \mathbf{u} = \frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} = 0.$$

Reconstructed fields from the diffusion prior violate this slightly.

Helmholtz projection

Any vector field $\mathbf{u}$ can be decomposed into divergence-free and curl-free parts. The divergence-free projection in Fourier space:

$$\hat{\mathbf{u}}^\perp(\mathbf{k}) = \hat{\mathbf{u}}(\mathbf{k}) - \frac{\mathbf{k}(\mathbf{k} \cdot \hat{\mathbf{u}}(\mathbf{k}))}{\|\mathbf{k}\|^2}, \quad \mathbf{k} \neq \mathbf{0}.$$

This removes the compressible component and has zero cost in Fourier space.

Integration into sampling

Projection is applied to $\hat{\mathbf{x}}_0$ *after* the data-consistency correction at each reverse step. No retraining. No additional trainable parameters.

DDNM + physics

After data-consistency correction, apply Helmholtz projection:

$$\hat{\mathbf{x}}_0^{\text{phys}} = P_{\text{div-free}}(\hat{\mathbf{x}}_0^{\text{corr}}).$$

Trade-off: hard projection may shift observed pixels. Fix: re-insert known pixels after projection, or project only the unknown part.

DPS + physics

Option 1 — add divergence as a soft guidance term:

$$\mathcal{L} = \mathcal{L}_y + \lambda_{\text{div}} \mathcal{L}_{\text{div}}, \quad \mathcal{L}_{\text{div}} = \|\nabla \cdot \hat{\mathbf{x}}_0\|_2^2.$$

Option 2 — project $\hat{\mathbf{x}}_0$ via Helmholtz after DPS correction (simpler, more stable).

RePaint + physics

Apply projection to the *generated* (unknown) part only, then re-insert the observed region:

$$\mathbf{x}_t = M \odot \mathbf{x}_t^{\text{known}} + (1 - M) \odot P_{\text{div-free}}(\mathbf{x}_t^{\text{generated}}).$$

Evaluation Protocol

Setup

For each held-out field $\mathbf{u}^{\text{true}}$:

- 1 Build observation $\mathbf{y} = A(\mathbf{u}^{\text{true}}) + \boldsymbol{\eta}$.
- 2 Run each sampler with the *same* checkpoint, operator, noise level, and seed.
- 3 Map reconstruction back to raw velocity space; compute metrics there.
- 4 Log one CSV row per run; save PNG visualisations for a subset.

Visualisation per sample

Ground truth $\mathbf{u}^{\text{true}}$ | Observation $\mathbf{y}$ | Reconstruction $\hat{\mathbf{u}}$ | Error map | Vorticity.

Scale

Debug run: $N = 16$, 1 seed.

Final table: $N = 256$, seeds $\{0, 1, 2\}$.

Evaluation Metrics

All metrics computed in **raw velocity space**.

Reconstruction accuracy

$$\text{Relative-}L_2 = \frac{\|\hat{\mathbf{u}} - \mathbf{u}^{\text{true}}\|_2}{\|\mathbf{u}^{\text{true}}\|_2}, \quad \text{RMSE} = \sqrt{\frac{1}{2HW} \|\hat{\mathbf{u}} - \mathbf{u}^{\text{true}}\|_2^2}.$$

Observation consistency

$$\text{Meas. error} = \frac{\|A(\hat{\mathbf{u}}) - \mathbf{y}\|_2}{\|\mathbf{y}\|_2}.$$

Physical consistency

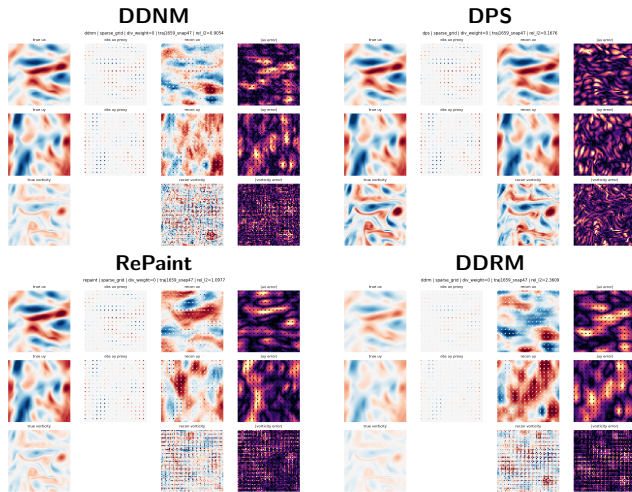
$$\text{Divergence} = \frac{\|\nabla \cdot \hat{\mathbf{u}}\|_2}{\|\hat{\mathbf{u}}\|_2}, \quad \text{Vorticity RMSE} : \omega = \partial_x u_y - \partial_y u_x, \quad \|\omega(\hat{\mathbf{u}}) - \omega(\mathbf{u}^{\text{true}})\|_2.$$

CSV columns

sample_id, seed, method, operator, noise_sigma, rel_l2, rmse, measurement_error, divergence, vorticity_rmse

Results: Sparse Grid Reconstruction

Regular sparse grid, stride 4 ($\approx 6\%$ observed pixels), noiseless.

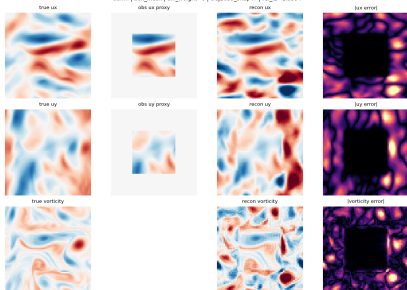


Results: Central Box Mask

Box mask 32×32 ; inner region known, outer region reconstructed. Noiseless.

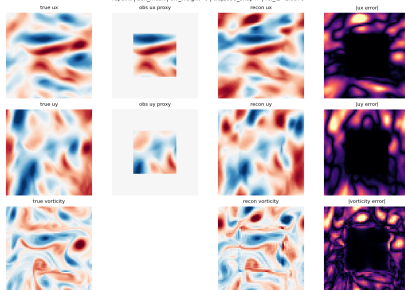
DDNM

ddnm | box_mask | div_weight=0 | traj(1659_snap47) | rel_2=1.3394



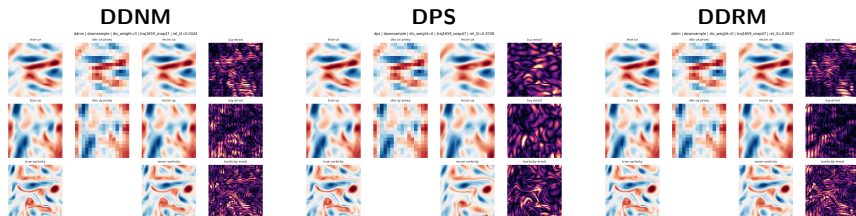
RePaint

repaint | box_mask | div_weight=0 | traj(1659_snap47) | rel_2=1.0678



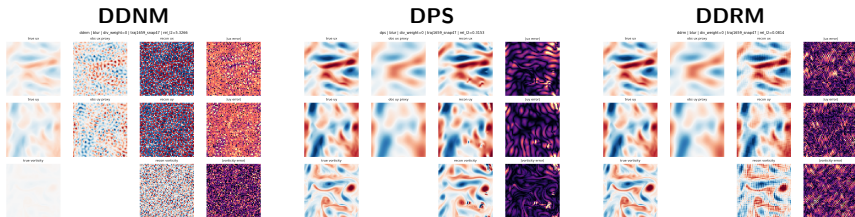
Results: Average Downsample

$64 \times 64 \rightarrow 16 \times 16$ average pooling ($4\times$ spatial reduction). Noiseless.



Results: Gaussian Blur

Periodic Gaussian blur with Gaussian observation noise.



Results: Quantitative Summary

Operator	Method	Rel.- L_2	RMSE	Meas. err.	Divergence
Sparse grid	DDNM	0.811	0.290	4.6×10^{-4}	0.523
Sparse grid	DPS	0.198	0.072	1.7×10^{-1}	0.038
Sparse grid	RePaint	1.188	0.426	1.9×10^{-8}	0.516
Sparse grid	DDRM	2.193	0.778	1.2×10^{-4}	0.516
Box mask	DDNM	1.515	0.546	9.9×10^{-5}	0.040
Box mask	RePaint	1.434	0.511	1.9×10^{-8}	0.035
Downsample	DDNM	0.050	0.019	5.9×10^{-5}	0.011
Downsample	DPS	0.303	0.107	2.7×10^{-1}	0.004
Downsample	DDRM	0.066	0.025	3.2×10^{-5}	0.020
Blur	DDNM	6.132	2.183	1.4×10^{-1}	1.958
Blur	DPS	0.430	0.155	1.1×10^{-1}	0.010
Blur	DDRM	0.079	0.030	1.3×10^{-5}	0.182

Benchmark summaries from the four Colab notebooks; rows use matching debug runs ($N = 16$).

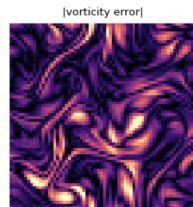
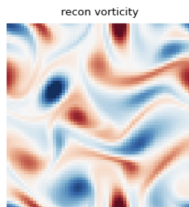
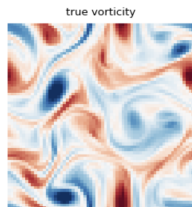
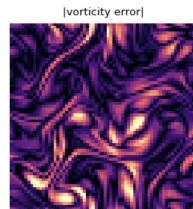
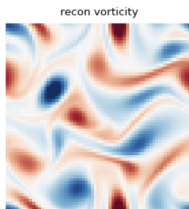
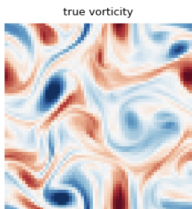
Physics Constraint: Does It Help?

Operator	Method	Div. base	Div. phys.	Rel.- L_2 base	Rel.- L_2 phys.
Sparse grid	DDNM	0.523	0.004	0.811	0.325
Sparse grid	DPS	0.038	0.026	0.198	0.198
Sparse grid	RePaint	0.516	0.058	1.188	0.501
Sparse grid	DDRM	0.516	0.095	2.193	2.174
Box mask	DDNM	0.040	0.004	1.515	1.479
Box mask	RePaint	0.035	0.009	1.434	1.022
Downsample	DDNM	0.011	0.001	0.050	0.049
Downsample	DPS	0.004	0.014	0.303	0.234
Downsample	DDRM	0.020	0.001	0.066	0.080
Blur	DDNM	1.958	0.041	6.132	4.999
Blur	DPS	0.010	0.007	0.430	0.455
Blur	DDRM	0.182	0.006	0.079	0.059

Helmholtz projection consistently reduces divergence for DDNM, RePaint, and DDRM. DPS is already close to divergence-free on the debug runs; its physics term mainly changes the reconstruction/data-consistency trade-off.

DPS: Physics Constraint — Visual Effect

Sparse grid operator, same sample. **Top:** with Helmholtz divergence-free penalty ($\text{div_weight} > 0$) — smooth, physically consistent reconstruction. **Bottom:** without physics constraint — visible block-like artefacts appear in the vorticity field where the gradient guidance overfits to measurement noise.



Summary

- Diffusion models provide a powerful data-driven prior for physical field reconstruction.
- Four zero-shot conditional sampling methods were implemented and compared across four observation operators on Kolmogorov flow.
- **DPS** is the most versatile: applicable to all operators, no pseudoinverse needed.
- **DDRM** is most suitable for spectral operators (blur, downsample) due to efficient per-frequency updates.
- **Helmholtz projection** provides a free improvement in physical consistency with minimal reconstruction penalty.

Limitations and future work

- All methods assume the score model is well-trained on in-distribution data.
- Physics projection is applied post-hoc; joint training of physics-aware score models could improve consistency further.
- Extension to 3D fields and time-dependent reconstruction is a natural next step.

Contributions

Mariia Abdullaeva	RePaint
Andrei Efimov	DDNM
Anton Losev	DDRM
Amir Sadreev	model training and DPS sampler

- 1 H. Chung, J. Kim, M. T. Mccann, M. L. Klasky, J. C. Ye. *Diffusion Posterior Sampling for General Noisy Inverse Problems*. ICLR 2023.
- 2 B. Kavar, M. Elad, S. Ermon, J. Song. *Denoising Diffusion Restoration Models*. NeurIPS 2022.
- 3 F. Rozet, G. Louppe. *Score-Based Data Assimilation*. NeurIPS 2023.
- 4 J. Song, C. Meng, S. Ermon. *Denoising Diffusion Implicit Models*. ICLR 2021.
- 5 A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, L. Van Gool. *RePaint: Inpainting using Denoising Diffusion Probabilistic Models*. CVPR 2022.
- 6 Y. Wang, J. Yu, J. Zhang. *Zero-Shot Image Restoration Using Denoising Diffusion Null-Space Model*. ICLR 2023.